

Relatorio da Etapa 2 - Sistema de Gestao Hospitalar

Projeto: Hospital Universitario Dra. Yuska Maritan Brito

Banco: PostgreSQL 18

Backend: Python, FastAPI e SQLAlchemy 2.x

1. Objetivo e evolucao do modelo

A Etapa 2 evoluiu a base relacional da Etapa 1 com regras de negocio no banco, consultas reutilizaveis, acesso por ORM e controle de concorrência. A migracao estrutural preserva os scripts historicos e adiciona os atributos necessarios ao novo enunciado: unidade do atendimento, horario de inicio do procedimento, media de tempo do procedimento e estado da supervisao. Tambem foram criadas as tabelas INTERNACAO e AUDITORIA_ATENDIMENTO.

Os registros existentes receberam backfill deterministico. A unidade de cada atendimento foi obtida da escala mais proxima do residente. Os procedimentos receberam horarios sequenciais, permitindo calcular o tempo entre a chegada do paciente e o primeiro procedimento. A migracao usa uma transacao e pode ser reaplicada sem duplicar objetos.

2. Triggers, procedures e views

Foram implementados tres triggers. O trigger de escala apresenta uma mensagem ligada a regra de negocio, enquanto a constraint unica continua sendo a garantia final contra corridas. O trigger de auditoria registra INSERT, UPDATE e DELETE com usuario, horario e estados antigo/novo em JSONB. O terceiro trigger recalcula a media observada de cada procedimento depois de uma nova execucao.

As procedures concentram operacoes completas. O cadastro de atendimento recebe um array JSONB de procedimentos e depende da atomicidade da transacao: se uma FK, CHECK ou chave unica falhar, atendimento, procedimentos, auditoria e medias sao revertidos. A procedure de espera devolve estatisticas por unidade. A procedure de reajuste bloqueia as escalas de origem, verifica o destino e atualiza data, turno e dia da semana.

A decisao entre trigger e procedure considerou responsabilidade. Triggers foram usados em invariantes que devem ocorrer independentemente do cliente, como auditoria e manutencao de media. Procedures foram usadas em fluxos explicitos, com parametros e resultado, como cadastrar um atendimento completo. As views encapsulam leitura: internacoes ativas, escalas sem supervisao valida e estatisticas mensais com procedimentos mais comuns.

3. ORM e consultas avancadas

Foi escolhido SQLAlchemy 2.x por integrar naturalmente Python, FastAPI e psycpg. Doze classes mapeiam as entidades e seus relacionamentos. As especializacoes PESSOA/PACIENTE/PROFISSIONAL e PROFISSIONAL/RESIDENTE/PRECEPTOR usam chaves primarias compartilhadas, sem alterar o modelo relacional.

Cada requisicao recebe uma Session. Escritas executam commit somente depois da operacao completa e rollback diante de falhas. Cadastros compostos sao persistidos em uma unica transacao por cascata ORM. As rotas nao possuem SQL textual: usam select, join, exists, group_by, having e funcoes de agregacao. Eager loading e usado quando a resposta sempre necessita dos relacionamentos; um endpoint separado demonstra a diferenca para lazy loading.

As consultas avancadas retornam: preceptores que supervisionaram residentes em atendimentos de pacientes flamenguistas; o ultimo atendimento de cada paciente com residente, preceptor e procedimentos; e o percentual de procedimentos de alto risco por residente. O percentual considera a quantidade executada, e residentes sem procedimentos permanecem no resultado com zero.

4. Concorrencia, validacao e conclusao

O cenario concorrente usa duas sessoes tentando escalar o mesmo residente na mesma data e turno. A aplicacao bloqueia a linha do residente com SELECT FOR UPDATE, pois uma escala de destino ainda pode nao existir. No teste, TX-B aguardou 1,219 segundo; depois do commit de TX-A, detectou o conflito e realizou rollback. O banco terminou com exatamente uma escala. Trigger e constraint complementam o lock para proteger clientes que nao utilizem o servico ORM.

A entrega possui testes transacionais para triggers, procedures e views, validacao estrutural de nove grupos de objetos, smoke test ORM e demonstracao concorrente com logs. O executor `run-phase-02.ps1 all` recria a base e executa todo o fluxo. A validacao final ocorreu em schema isolado e terminou sem residuos. Assim, os requisitos obrigatorios da Etapa 2 foram implementados e demonstrados de forma reproduzivel.